The PINN Dielectric Project, Explained Simply

Oct 17, 2026 ·

1. The big question, and the short answer

Question: can a new kind of AI-based equation solver, called a PINN, correctly calculate the electric field where two different insulating materials touch?

Short answer: in the simplest flat, one-dimensional case, some versions can, up to quite large differences between the materials. In realistic two-dimensional shapes (a rod, a sharp corner, a thin layer), every version we tried got the field badly wrong once one material was 80 times more "polarisable" than the other. The traditional method, the finite element method (FEM), got within 1% in under 5 seconds.

Why this matters: the strongest electric fields, where insulation breaks down and sparks happen, are found exactly at these boundaries. A solver that gets the boundary wrong is useless for safety design, even if it looks correct everywhere else.

What we did, in one line each:

1. Picked 5 test problems, from a simple capacitor to a thin film.
2. Picked 6 different ways ("recipes") of setting up a PINN.
3. Trained each recipe on each problem at many material-contrast values, 5 times each with different random starts.
4. Measured the error in the electric field right at the boundary.
5. Tried 5 fixes, one at a time, under fair conditions.
6. Checked that the conclusions do not depend on small choices.
7. Raced the PINNs against FEM for speed and accuracy.
8. Wrote down every rule before running, and every change and mistake as it happened.

In total about 3,900 training runs were recorded.

2. The physics you need (Class 12 electrostatics)

Everything here is ordinary Class 12 electrostatics; the only new idea is what happens at the boundary between two dielectrics.

Potential and field. The electric potential V (we write φ) is like height on a hill. The electric field E is how steep the hill is, pointing downhill:

$$E = -\frac{dV}{dx}$$

Dielectrics. An insulator placed in a field gets polarised: its molecules stretch slightly, creating opposite charges on its surfaces. This weakens the field inside it. How strongly a material does this is its permittivity ε, written as $\varepsilon = K\,\varepsilon_0$, where K is the dielectric constant you met with capacitors (K = 1 for vacuum, about 80 for water).

Electric displacement D. Physicists use $D = \varepsilon E$. You can think of D as "the field created by the free charges only". Gauss's law says the flux of D through any closed surface equals the free charge inside.

The key example: a capacitor with two layers. Take a parallel-plate capacitor filled half with material 1 (constant K_1) and half with material 2 (K_2). This is exactly two capacitors in series, as in your textbook. There is no free charge at the boundary, so:

1. **D is the same in both layers** (Gauss's law with a thin pillbox across the boundary).
2. **So E is different:** $E_1 = D/\varepsilon_1$ and $E_2 = D/\varepsilon_2$. If material 2 has $K_2 = 80\,K_1$, the field in it is 80 times weaker.
3. **V is continuous:** no sudden jump in voltage at the boundary, but its slope changes suddenly. The graph of V has a sharp bend (a "kink").

We call the ratio $\kappa = \varepsilon_2/\varepsilon_1$ (Greek letter kappa) the **contrast**.

A real number from our project. With κ = 80, 1 volt across the plates and the boundary in the middle, the program gives D = 1.975 (in scaled units), $E_1 = 1.975$ and $E_2 = 0.0247$. The voltage falls from 1 V to 0.506 V within the first quarter of the gap; almost all of the voltage drop happens in the low-K layer. That bend is what the solvers must capture.

The general equation. In more complicated shapes we cannot use the series-capacitor formula, so we solve Gauss's law written as an equation at every point:

$$\nabla \cdot (\varepsilon \nabla V) = -\rho$$

At each boundary between materials, the same two conditions as in the capacitor must hold: V continuous, and the perpendicular part of D continuous. These are the **interface conditions**.

3. How a computer solves it: FEM versus PINNs

Two very different approaches were compared: the well-established finite element method, and the newer physics-informed neural network.

The finite element method (FEM). Cut the region into thousands of small triangles (a "mesh"). Inside each triangle, approximate V by a simple polynomial. Gauss's law then becomes a large set of linear equations, like the simultaneous equations you solve in

maths, but with tens of thousands of unknowns, solved by a standard computer routine. If the triangle edges are laid along the material boundary (an *interface-conforming* mesh), the bend in V is built in automatically. Making the triangles smaller makes the answer more accurate in a predictable way.

A neural network. A neural network is a flexible mathematical function with many adjustable numbers called *weights*. Ours take the position (x, y) as input and output a value of V. Inside, each "neuron" multiplies its inputs by weights, adds them, and passes the result through a smooth S-shaped curve (tanh). We used 3 layers of 32 neurons, about 2,200 weights per network. Changing the weights changes the function, like adjusting many knobs at once.

A physics-informed neural network (PINN). Normally a neural network learns from examples of right answers. A PINN instead learns from the *laws of physics*. Training repeats these steps:

1. Choose a few hundred sample points in the region.
2. Ask the network for V at those points, and calculate its slopes and curvatures exactly by a technique called automatic differentiation.
3. Check how badly the network breaks the rules: Gauss's law at each point, the fixed voltages on the plates, and (in some recipes) the interface conditions.
4. Add up the squares of all these violations into one number, the **loss**. A perfect solution has loss zero.
5. Nudge every weight slightly in the direction that lowers the loss (the slope of the loss with respect to each weight is called the *gradient*; the method is called gradient descent; the variant we used is called Adam).
6. Repeat. We used 3,000 repetitions ("steps") per training run.

Why people like PINNs. No mesh is needed, the answer is a smooth formula you can evaluate anywhere, and the same idea also works for "inverse" problems (working backwards from measurements). Our question is whether they are accurate at material boundaries.

Why boundaries are hard for PINNs. Two reasons:

- A smooth network has difficulty making a sharp bend; it naturally learns gentle shapes first.
- More seriously, inside one material ε is constant, so Gauss's law only says V has no curvature there. Unless the recipe adds the interface conditions to the loss, *nothing in the loss tells the network how the two sides must join*. A wrong answer can then have zero loss.

4. The five test problems

The five problems go from a textbook capacitor to shapes that occur in real devices; each adds one new difficulty.

Code	Picture	Real-life example	What makes it hard
P1	Two-layer parallel-plate capacitor (1D: only depends on distance between plates)	Layered insulation, capacitor with two dielectrics	Just the kink at one flat boundary; the exact answer is known (straight lines)
P2	A dielectric rod (circle in 2D) placed in a uniform field	Fibre in a composite, cable insulation	The boundary is curved
P3	A line charge (a long charged wire) above a flat boundary	Charged conductor near a dielectric surface	The field becomes infinite at the wire (a "singularity")
P4	A corner where one material fills a quarter of the plane	Edge of an electrode or insulator	The field becomes infinite at the corner tip
P5	A very thin high-K film (thickness 0.02, width 1) in a uniform field	Thin coating, capacitor dielectric layer, chip insulation	The film is 50 times thinner than it is wide

For P1 to P4, the exact answer is known from formulas (P1 from the series-capacitor result, P2 from the classic dielectric-cylinder result, P3 from the method of images, P4 from a corner formula). For P5 there is no formula, so a very fine and carefully checked FEM solution was used as the "true" answer.

Each problem was solved at many contrasts: $\kappa = 1$ (both materials the same, a sanity check), 5, 20, 80, 400 and 2000. In 2D we also tried the reverse case, κ less than 1, where the outer material has the higher permittivity (down to $\kappa = 0.0005$).

5. The six PINN recipes (F1 to F6)

The six recipes differ in two things: how the network is built, and which rules go into the loss.

Recipe	How it is built	What the loss checks	Analogy
F1	One network for the whole region	Gauss's law in each material + plate voltages. Nothing about the boundary	A student told the rules for each room but not how the rooms connect
F2	Two networks, one per material	Same as F1, plus penalties for a voltage jump and for a D jump at the boundary	Two students, one per room, fined if their answers don't match at the door
F3	One network that is also told "how far am I from the boundary"	Gauss's law, plate voltages, D jump	A student given a map showing where the door is
F4	Two parts glued so that V is automatically continuous and the plate voltages are automatically right	Gauss's law + D jump	Some rules made impossible to break by construction
F5	Like F3, with plate voltages built in	Minimise the total stored energy $\frac{1}{2}\int \varepsilon E^2$ instead of checking Gauss's law (nature's "least energy" principle)	Find the lowest valley rather than check every slope
F6	Same as F2	Same as F2, but each penalty divided by its natural physical size so no term dominates	F2 with a fairer marking scheme

All used the same small networks, the same training method (Adam, 3,000 steps), the same number of sample points, and the same computer settings. F3, F4 and F5 were run at fewer contrasts in 2D (κ = 1, 80, 2000) to save time; this was decided in advance.

6. How success was measured, and how we kept it fair

A run **failed** if its electric field near the boundary was more than 5% wrong; every recipe was run 5 times, and all rules were written down before the experiments.

The interface band. We looked at a thin strip within 0.05 units of the boundary (the "band"), because that is where breakdown happens and where solvers struggle.

The error measure. In the band we compared the computed E and D with the true values point by point and took a root-mean-square difference, divided by the size of the true values (a "relative L2 error"). An error of 0.05 means 5% wrong; an error of 1 means 100% wrong. The larger of the E error and the D error is the **primary error**.

The 5% rule. Primary error above 5% means the run failed. For comparison we also measured the error in V over the whole region (the **global potential error**), which is what most published papers report.

Why five repeats ("seeds"). A neural network starts from random weights, and different random starts can give quite different answers. Each set-up was trained 5 times with 5 different random starts, and we report the middle value (median) and how many of the 5 failed. Reporting only the best of several attempts, which some papers do, can make a method look much better than it usually is.

A reality check on "5 repeats". 0 failures out of 5 sounds perfect, but statistically it is still consistent with a true failure rate as high as about 50%. So small numbers are reported honestly as small numbers.

Pre-registration. Before running the main experiments we wrote down, in a dated document:

- 11 predictions ("hypotheses" H1 to H11);
- exactly how each would be judged supported or refuted;
- all the settings.

This stops anyone (including us) from changing the rules after seeing the results. Whenever a rule had a gap, the decision was written into a **change log** with the date and reason before the affected experiments were run.

Checking the checker. Before trusting the measuring program (the "evaluator"), we tested it: fed it exact answers (it must report zero error), deliberately damaged answers (it must report the right error), and so on. It passed 98 out of 98 checks in 1D and 17 out of 17 for each 2D problem.

7. Results on the simple capacitor (1D)

On the two-layer capacitor, the two-network recipes F2 and F6 coped with contrasts up to 400–2000, while F1 failed as soon as the materials differed.

Number of failed runs out of 5:

Recipe	κ = 1	5	20	80	400	2000
F1	0	5	5	5	5	5

Recipe	κ = 1	5	20	80	400	2000
F2	0	0	0	1	2	5
F3	0	0	4	5	5	5
F4	0	0	5	5	5	5
F5	0	0	5	5	5	5
F6	0	0	0	0	0	3

At κ = 1 (no boundary at all) every recipe works, which confirms the code is correct. At an even higher test contrast, κ = 10,000, F2 and F6 also failed all 5 runs.

Why F1 fails: it is blind. F1′s loss contains no information about how the two materials join. The answer "pretend both layers are the same material" (a single straight line) satisfies every rule in F1′s loss. F1 found exactly that wrong answer: at κ = 80 its field error was 2,780%, and our check of the effective contrast showed it behaved as if κ were 1. This was known from earlier research; we measured how bad it is at every contrast.

Why F2 eventually fails: the wrong rule gives way. We expected the D-matching rule to break first. It was the other way round: in failing runs, D matched almost perfectly (mismatch about 0.03%), but V had a small jump at the boundary (about 5% of the applied voltage). At high contrast the D penalty is numerically huge, so training satisfies it first and leaves the voltage slightly broken. Prediction H3 ("the D rule fails") was therefore **refuted**.

Example of seed-to-seed variation. F2 at κ = 400 gave errors of 21%, 8.2%, 0.5%, 1.1% and 4.8% for the 5 random starts. Showing only the best (0.5%) would hide the two failures.

A surprise. The recipes that build the boundary into the network (F3, F4, F5) failed earlier (from κ = 20) than plain F2 and F6. With more training (or a better optimiser, Section 10) F3 does succeed, so their problem is that they are harder to train within 3,000 steps.

8. Results on realistic shapes (2D)

In two dimensions every recipe failed every run once κ reached 80, on all four shapes; the 1D success did not carry over.

Median field error at κ = 80 (1 = 100% wrong):

Problem	F1	F2	F3	F4	F5	F6
P2 rod	30	0.63	1.0	0.84	1.2	0.63
P3 line charge	31	6.1	3.5	8.5	13,000	11
P4 corner	14	0.69	1.0	17	1.7	0.70

Problem	F1	F2	F3	F4	F5	F6
P5 thin film	30	0.070	1.6	2.4	2.8	0.069

Even the best entries (F2 and F6 on the thin film, 7% error) are above the 5% limit.

The only 2D successes away from κ = 1:

- the thin film P5 with F2 or F6 at κ = 5 (no failures) and κ = 20 (1–2 failures out of 5);
- the line-charge problem P3 with F2 at mild reverse contrasts (κ = 0.2 and 0.05).

In 1D, F2 and F6 first failed at κ = 2000; on the rod, the line charge and the corner they already failed at κ = 5. The prediction that results in 1D would predict results in 2D (H8) was **refuted**.

What we learned about each shape:

- **Rod (P2): stuck, not slow.** F2 and F6 got stuck at about 63% error. A run 7 times longer, with double-precision numbers, stayed stuck at 49% from step 4,000 to step 20,000. We checked that the loss itself is correct: when given a simple family of functions that includes the exact answer, minimising F2′s loss finds the exact answer to 6 digits. So the trouble is in training the network, not in the rules.
- **Line charge (P3):** removing the infinite part of the field separately in each material (an extra test planned in advance) cut F2′s error from 610% to 61%, but it still failed.
- **Corner (P4):** we predicted that smooth networks would under-estimate the field near the corner tip (H10). F2 did under-estimate it, but F1 over-estimated it, by up to 1,860%, in the high-K wedge. The direction of the error depends on the recipe, so H10 was **refuted**.
- **Thin film (P5):** a thinner film *looked* easier (band error 13% → 7% → 3.4% as thickness went 0.1 → 0.02 → 0.005), but the error *inside* the film stayed at 13–16% for every thickness; the band average was simply dominated by the easier field outside. Using 4 times more sample points in the film changed nothing (6.9%, 7.0%, 6.9%).
- **The energy recipe F5 "cheats":** it estimates the stored energy by averaging over a fixed set of sample points. On P2 it found answers whose estimated energy (0.6–1.6) was *below* the true minimum possible energy (2.0), which is impossible for a real field. The network had learned to look good at those particular points while being wrong in between, much like memorising the answers to the exact questions on a practice test.

9. Hidden failures: when "looks right" is wrong

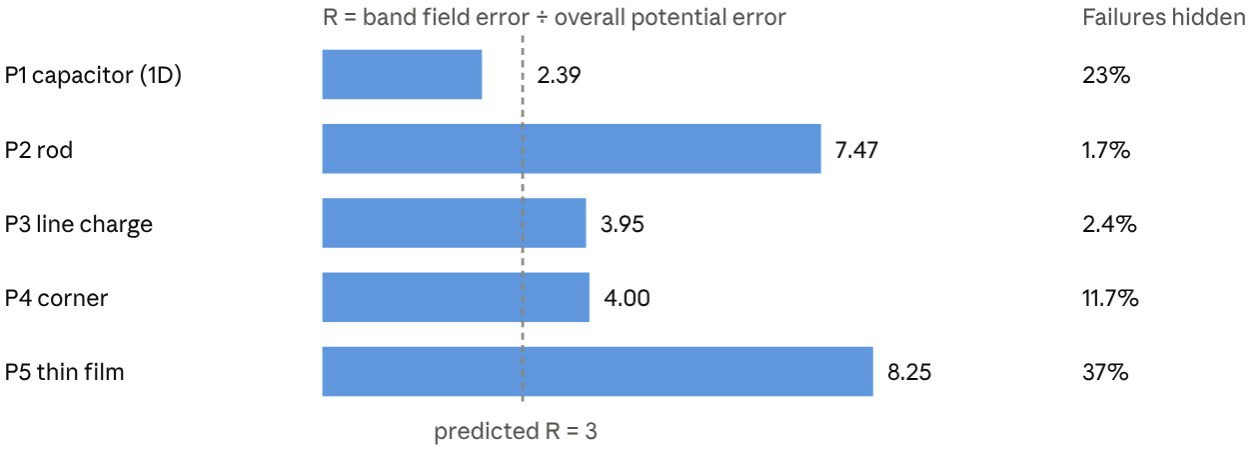
The usual overall error (in V, over the whole region) hides how wrong the field at the boundary is: the boundary field error was typically 2.4 to 8 times larger.

Why this happens. The field is the slope of V. A curve can be very close to the right curve in height, yet have quite the wrong slope in one small place, like a road that reaches the right town but has a dangerously steep bend that the map does not show. Most published PINN papers report only the overall error in V.

We call the ratio R = (boundary field error) ÷ (overall V error). A **hidden failure** is a run that fails our 5% field test while its overall V error is below 5%, so it would be reported as a success by the usual measure.

The boundary field error is several times the overall potential error

Median ratio R from 2.4 to 8.3, over all runs with κ ≠ 1



Project database, all main runs with κ ≠ 1; "failures hidden" = failing runs whose overall potential error is below 5%

On the thin film, more than a third of all failures (60 of 163) would have looked like successes. Our prediction H2 said R would be above 3; on the 1D capacitor it was 2.39, just short, so H2 counted as **inconclusive** there, but it held on every 2D shape.

Lesson: always measure the quantity you actually care about, in the place where you care about it.

10. Five attempted fixes, tested fairly

Of five popular fixes, only a better optimiser (L-BFGS) clearly helped, and only on the simple capacitor; one apparent improvement turned out to be our own mistake.

How the tests were made fair. Like a controlled experiment in a chemistry lab, each test changed exactly one thing and kept everything else the same, including the same 10 random starts. For each start we compared "with fix" and "without fix" directly. A fix counted as working at a setting if it helped in at least 9 of the 10 starts. These tests were run on the 1D capacitor.

Fix	Idea in plain words	Result
A. Automatic re-weighting	Every 10 steps, turn up the penalties whose gradients are small so all rules get equal attention	**Mixed.** Helped F3 at high contrast (error 106% → 3.3% at κ = 400) but made F2 fail every run even at κ = 1
B. Rescaling the equation	Divide Gauss's law in each material by its ε so both materials' terms are the same size	**Did not help** (improved only 1–2 of 10 starts)
C. More precise arithmetic	Use 64-bit ("double") numbers instead of 32-bit	**No effect** with the usual optimiser; a clear benefit with L-BFGS
D. Build continuity in	F4 forces V to be continuous; F4full forces every boundary condition exactly	F4 **did not help** (worse than F2). F4full was excellent, but only because in 1D the exact answer is simple enough to build in
E. A smarter optimiser	L-BFGS, which uses curvature information to take better steps, with the same number of calculations	**Mostly helped.** F3 at κ = 80: 55% → 0.0035% error; F2 failures at κ = 2000 dropped from 9 to 4 of 10. But F5 got worse at κ = 1, by "cheating" on its sample points again

Why fix A backfired. Its rule gives a big weight to any rule whose gradient is small. A rule that is already satisfied has a small gradient, so it got weights up to 50,000, while the rule that was still broken (voltage continuity) was turned down to 0.05. The fix pushed attention away from the actual problem. We confirmed this by writing the method a second, independent way and getting the same numbers.

The double-precision story. At first, 64-bit arithmetic appeared to change the error by up to 20 times. We investigated and found a bug: the 64-bit runs had accidentally started from *different random weights* than the 32-bit runs. Once both started from identical weights, the difference almost vanished (at most 7%). So the big "effect" was simply a different random start, which shows how much the random start alone matters and why repeating runs is essential.

Prediction H4 said F2's high-contrast failures were just a training problem that fixes A or B would cure. Neither did, so H4 was **refuted** by its own test. Fix E suggests that training is part of the story in 1D.

11. Robustness: would different settings change the story?

The order of the recipes, from worst to best, stayed essentially the same under all 8 changes of settings we tried, so the ranking is not an accident of our choices.

A fair critic might say: "maybe you just chose a network that was too small, or trained it too little". So on the 1D capacitor we re-ran everything with one setting changed at a time:

- narrower network (16 neurons) or wider network (64);
- fewer layers (2) or more layers (5);
- fewer sample points (32 per material) or more (512);
- a faster learning rate;
- 4 times longer training (12,000 steps).

In every case F1, F3, F4 and F5 failed most, and F2 and F6 failed least. We measured agreement with a standard ranking statistic (Kendall's τ, where 1 means identical order); it was 0.65 or higher every time, above our threshold of 0.6.

Interesting details:

- **Longer training helps F2 but makes hidden failures more common.** With 12,000 steps F2's failures fell from 6 to 1 (out of 15), but hidden failures doubled (7 → 16): training improved the overall V more than the field at the boundary.
- **Network size matters only for F2 and F6.** A wider network brought F6 to zero failures; a narrower one raised F2 from 6 to 9.
- **It is not because F2 and F6 have more weights.** They use two networks, so we also gave them smaller networks with fewer total weights than F1; they still did far better.

The hidden-failure ratio R stayed around 2 (1.8–2.5) under 7 of the 8 changes. Prediction H6 needed R above 3 as well, so it was judged **inconclusive**: the ranking is robust, and the hiding effect is steady but smaller than we predicted.

12. The race: PINNs versus FEM

FEM reached 1% field accuracy in under 5 seconds on the rod and the thin film; no PINN reached 1% at any contrast other than $\kappa = 1$, even with the best fixes.

We gave the PINNs their best chance: besides the normal recipes, we added the two fixes that helped on the capacitor, L-BFGS (90 extra runs) and training 6.7 times longer (20 extra runs). Then we asked two questions.

Question 1: how long to reach 1% (and 5%) error? For FEM, we took the coarsest mesh that was accurate enough, and timed building the mesh plus solving. For PINNs, we checked their progress every 250 steps.

Example, the rod (P2) at κ = 80:

Method	Field error in the band	Time
FEM, 565 unknowns	17.6%	0.01 s
FEM, 7,869 unknowns	3.0%	0.13 s
FEM, 30,337 unknowns	1.0%	0.67 s
FEM, 119,697 unknowns	0.4%	4.4 s
PINN F2, normal training	63%	10 s
PINN F2, L-BFGS	64%	21 s
PINN F2, 20,000 steps	64%	95 s

On the rod, neither fix moved F2 at all: it is stuck, not slow.

Question 2: at the same computing time, who is more accurate? Even FEM's finest mesh took less time than any PINN, and it was 86 to 170,000 times more accurate (away from κ = 1).

The PINNs' best moment. On the thin film at κ = 80, L-BFGS got F2 and F6 just under 5% (4.9%) in about 5 seconds. FEM reached 5% in 0.04 seconds, about 115 times faster.

Prediction H7 (FEM reaches 1% at least 10 times faster and is more accurate at equal time) was **supported**. At κ = 1 the comparison is unfair to PINNs in a different way: there is no boundary, so FEM is exact immediately.

Fair caveats. Times were measured on an ordinary computer processor (CPU), one core; a graphics card (GPU) would speed up PINN training, but would not change the fact that they never reached 1%. And FEM is not always easy: for the line charge at κ = 2000, even FEM with 120,000 unknowns only got to 12%.

13. Mistakes we found and fixed

Six programming mistakes were found during the project; each was explained, fixed, given an automatic test so it cannot return, and the affected results were re-run, with the old results kept and labelled.

Scientists make mistakes; what matters is catching them and being open about them. For each mistake we followed the same five steps: explain it, fix it, add a test that would catch it again, find which results it affected, and re-run those, keeping the originals visible.

Mistake	What went wrong	Effect and what we did
FEM field calculation (2D)	A formula slip in computing E from the FEM solution	Caught by a test before any results used it
Line-charge training	A program crashed when the number of sample points changed between runs	60 runs failed for a technical reason; kept on record, fixed, re-run
Corner checking code	An error in a derivative in the code that checks the corner formula	Affected only the checking code; no results changed
64-bit precision test	64-bit runs started from different random weights than 32-bit runs	Made precision look important when it was not; all 270 runs re-run
Energy recipe with extra points	F5 averaged its energy over points that were bunched near the boundary, giving that region double weight	15 runs re-run with correct weighting; F5 then behaved normally
Summary script	One script mixed data from other problems into the rod's table when re-run	Caught when we checked that every table could be rebuilt from raw data; fixed

We also **re-checked the final report against the raw data** and corrected our own writing errors: for example, an early draft said "5,300 runs"; the database actually holds 3,856 completed runs, and the text was corrected.

Reproducibility. One command rebuilds every table and figure from the raw data in about 70 seconds, and an automatic test re-trains stored runs to check they come out identical (to 13 decimal places). In total 169 automatic tests pass.

14. What the project shows, and what it does not

The results hold for these 5 problems, these 6 recipes, small networks and a fixed training budget; they do not prove that PINNs can never work at boundaries.

What it shows:

1. A single-network PINN with only Gauss's law in its loss (F1) cannot handle a dielectric boundary at all: its rules are satisfied by a wrong answer.
2. Two-network recipes (F2, F6) work on a flat 1D boundary up to high contrast, but on curved, corner, line-charge and thin-film shapes every recipe failed once κ reached 80.

3. When they fail, it is usually the voltage-continuity rule that gives way, not the D rule.
4. The usual overall error hides many failures; on the thin film more than a third of failures would have looked like successes.
5. Several popular fixes did not help when tested fairly; the best fix (L-BFGS) helped in 1D but not on the rod.
6. FEM with a mesh that follows the boundary was faster and more accurate in every 2D comparison.
7. Random starting points matter so much that a single run, or the best of a few, can be very misleading.

What it does not show:

- That PINNs cannot solve boundary problems: much larger networks, specially designed boundary networks, adaptive sample points or much longer training were not tested at scale, and some published methods report success.
- That FEM is always better: FEM also struggled near the line charge, and PINNs have uses we did not test, such as working backwards from measurements.
- Anything about 3D, time-varying or non-linear problems.

How the 11 predictions turned out: 4 supported (H1, H7, H9, H11, though H11 is mixed up with contrast), 4 refuted (H3, H4, H8, H10), 2 inconclusive (H2, H6), and H5 was judged fix by fix (mixed). Refuted predictions are useful results too: they tell other scientists which easy explanations are wrong.

Take-home lessons for any scientist: measure what matters where it matters; repeat experiments with different random starts; write your rules down before you look at the results; change one thing at a time; and keep a record of every mistake.

15. Glossary

Term	Meaning
Adam	The standard step-by-step method used to adjust a network's weights
Automatic differentiation	A way for the computer to calculate exact derivatives of the network's output
Band (interface band)	The thin strip within 0.05 units of the boundary where errors were measured
Contrast κ	Ratio of permittivities, $\varepsilon_2/\varepsilon_1$; κ = 1 means both materials are identical

Term	Meaning
D (electric displacement)	εE; its perpendicular part is continuous across an uncharged boundary
Dielectric	An insulating material that becomes polarised in an electric field
Step	One update of all the weights; we used 3,000 steps
FEM (finite element method)	Traditional solver that divides the region into small triangles
Gradient	How the loss changes when each weight changes; tells training which way to go
Hidden failure	A run that fails the field test while its overall V error looks small (below 5%)
Hypothesis (H1–H11)	A prediction written down before the experiment, with a pass/fail rule
Interface conditions	V continuous and perpendicular D continuous at a material boundary
Interface-conforming mesh	An FEM mesh whose triangle edges lie along the material boundary
L-BFGS	A smarter optimiser that also uses curvature information
Loss	One number measuring how badly the network breaks the rules; zero = perfect
Median	The middle value when results are sorted; not affected by one extreme run
Neural network	A flexible function with many adjustable weights
Permittivity ε	How strongly a material polarises; $\varepsilon = K\varepsilon_0$
PINN	Physics-informed neural network: a network trained to obey physical laws instead of copying answers
Pre-registration	Writing down predictions and rules before running the experiments
Primary error	The larger of the relative E and D errors in the band; above 5% = failure

Term	Meaning
R (masking ratio)	Band field error ÷ overall V error
Recipe (F1–F6)	A particular way of building the network and choosing its loss
Relative L2 error	Root-mean-square difference divided by the size of the true values
Seed	The number that fixes the random starting weights and sample points of a run
Singularity	A point where the field becomes infinite (wire, sharp corner)
Weights	The adjustable numbers inside a neural network